PDCT IS STILL A NEW APPROACH

As of writing this book, PDCT is still a fairly new approach, introduced to the public in 2022. This can be seen as a drawback because support and resources might not be as widespread as they are for CDCT. However, PactFlow's documentation is always a good start.

RISK OF FALSE CONFIDENCE

In the PDCT approach, the consumer contract isn't tested against the provider code but against the contract that the provider uploads. PactFlow depends on the accuracy of these contracts to inform you if they are compatible. Because PDCT relies on two contracts being uploaded, there is a risk of false confidence if the contract uploaded by the consumer reflects only some of the expected consumer interactions or if the provider doesn't provide an up-to-date API specification. When this happens, you can run into scenarios where breaking changes might slip to production.

11.2 Implementing consumer test adapters

Generating contracts from consumer test mocks, Cypress and WireMock have been chosen to demonstrate the Pact adapters functionality. The adapter packages are installed as part of the test project and offer custom commands or functions to convert mocks to Pact contracts. Using existing mocks is a great way to get started with contract testing.

11.2.1 Cypress consumer adapter

In this example, we'll be using Cypress to generate our contracts from mock responses. We'll use the Cypress `intercept` command to return the mock response. Then, we'll use Pact's `wait` command to wait for the API request to be made and generate the contract from the intercepted mocked response.